

# Land Cover Classification in the Delhi-NCR Region

---

**Name:** Vaidehi Bajaj ( <https://sites.google.com/ds.study.iitm.ac.in/vaidehibajaj> )

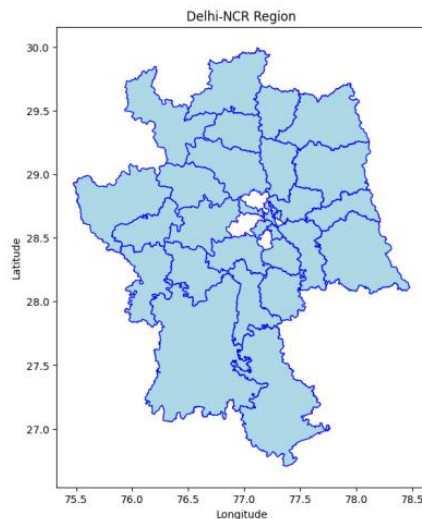
**Objective:** This task demonstrates a workflow for acquiring and processing geospatial data, extracting land cover patches, and developing a convolutional neural network (CNN) model to classify these patches within the Delhi-NCR region. The task is divided into three main problems, covering data acquisition, data preparation, and model development/evaluation.

## Problem 1: Data Acquisition and Visualization

This problem focused on acquiring and visualizing the geographic areas of interest: the Delhi-NCR region and the Delhi airshed.

### 1.1: Visualization of Delhi-NCR Region:

The delhi\_ncr\_region.geojson file was loaded and visualized. The generated map showed the boundary of the Delhi-NCR region.



### 1.2: Visualization of Delhi Airshed:

The Delhi airshed boundary was loaded into Google Earth Engine (GEE) this involved creating a Google Cloud project in the Google Cloud console, and bringing the project ID to initialize GEE.

It was then visualized interactively over a satellite basemap and the Delhi airshed boundary was overlaid in yellow. This map allows for exploration of the airshed's location relative to the underlying satellite imagery.



### 1.3-1.5 : Downloading Sentinel-2 images and Visualization of Sample Downloaded Tiles:

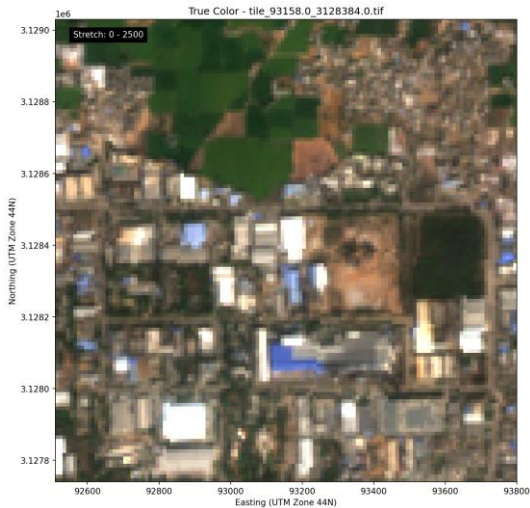
The next step was an attempt to download Sentinel-2 image tiles covering the Delhi airshed from Google Earth Engine to Google Drive. This process involved defining a grid over the airshed, filtering Sentinel-2 imagery for a specific time period and cloud cover, and submitting export tasks for each grid tile.

An attempt to download all 4373(total count of images) Sentinel-2 image tiles covering the airshed was interrupted due to GEE task limits, and the full dataset download was not completed. Data processing proceeded using a smaller, acquired dataset of about 600 images.

Folder link containing sample downloaded images-

<https://drive.google.com/drive/folders/1SINtA3zQsqgCAtg9AFmobl-DCcX6ehvq?usp=sharing>

Following the download, a few sample downloaded tiles were visualized to check their content. These visualizations showed true-color composites of the satellite imagery and these visualization confirmed that some image data was available and could be processed further(image of 1 sample is attached below).



### 1.6: While downloading satellite imagery some important parameters to be considered are :

- Geographic Extent (AOI): Precisely defines the spatial area of interest to be covered.
- Time Period: Specifies the date range to acquire relevant imagery.
- Cloud Cover: A critical filter to ensure data usability and quality by limiting cloud obscuration.
- Spectral Bands: Selects only the specific spectral information (e.g., RGB) needed for the analysis, reducing data size.
- CRS (Coordinate Reference System): Ensures the output is correctly georeferenced and projected
- .Export Region: Defines the exact geometry for each individual file, essential for tiling large areas.
- GEE Task Limit: Crucial for managing concurrent exports, preventing the system from hitting the platform's API limits.
- Batch Size and Sleep: Implements controlled submission and pauses between batches to manage server load and improve the reliability of mass exports.

## Problem 2: Land Cover Patch Extraction and Preparation

This problem focused on extracting small image patches from the land cover data and preparing them for training a classification model. The key steps involved:

### 2.1-2.3 : Patch Extraction & Labeling

- A 128 \* 128 pixel patch is extracted from the land\_cover.tif (ESA WorldCover) centered on the coordinates of each Sentinel-2 tile, handling CRS transformations as needed.
- The patch's label is assigned using the mode (most frequent) land cover class within the patch.
- Raw ESA codes are mapped to 11 standardized land cover labels .

### 2.4 : Edge Case Handling & Treatment

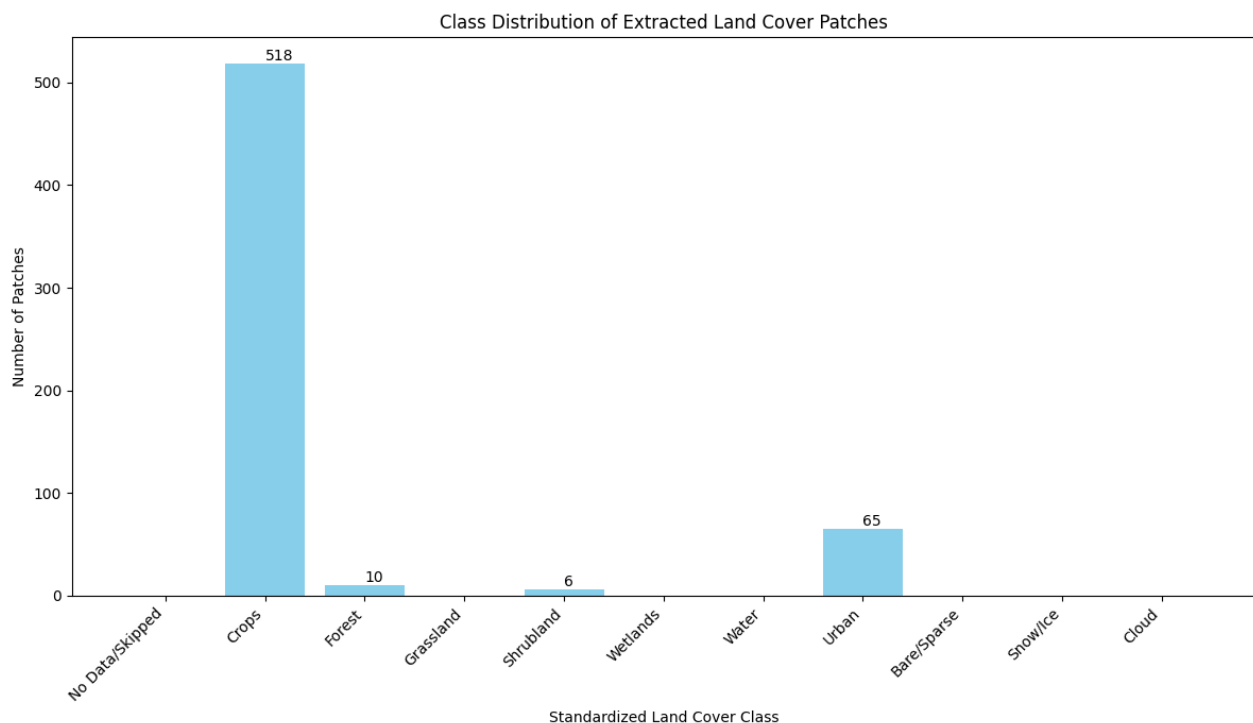
- No Data Pixels (0): These are excluded from the mode calculation; the default label (0) is assigned if no valid pixels exist or if dominance is minimal.
- Mixed Class Dominance: Patches where the mode count is only 1 (low confidence) are assigned the default label (0).

### 2.5-2.6 : Dataset Split & Visualization

- The final dataset is split 60% (Train) / 40% (Test) using stratified random splitting to maintain class proportions.
- Visualization (bar chart) is performed to show the class distribution.

### Observation on Data Distribution:

After processing the tiles and extracting patches, the distribution of the standardized land cover classes in the extracted dataset was visualized using a bar plot.



The visualization and the output showed a significant class imbalance:

- The 'Crops' class accounted for a very large majority (over 86%) of the samples.
- This observation is crucial as it indicates that standard accuracy might be misleading when evaluating model performance.

### Problem 3: Model Development and Evaluation

This problem focused on building, training, and evaluating a Convolutional Neural Network (CNN) model for land cover classification using the extracted patches.

#### 3.1: Model Training and Architecture

- A simple CNN model with convolutional and fully connected layers was defined in PyTorch for 11 land cover classes.
- Training data was converted to PyTorch tensors and loaded in batches
- CNN Training: A SimpleCNN was trained for 10 epochs using Cross-Entropy Loss and the Adam optimizer. The loss decreased from approx. 14.77 to 0.0425.
- Architecture: The SimpleCNN uses two Conv2d/ReLU/MaxPool2d blocks (3x3 kernel size, 1-channel -> 32 -> 64 channels), followed by flattening and two Fully Connected layers (65536 -> 128 -> 11 classes).

```
Epoch [1/10], Loss: 14.7742
Epoch [2/10], Loss: 1.1396
Epoch [3/10], Loss: 0.4881
Epoch [4/10], Loss: 0.3200
Epoch [5/10], Loss: 0.1804
Epoch [6/10], Loss: 0.1675
Epoch [7/10], Loss: 0.1145
Epoch [8/10], Loss: 0.1477
Epoch [9/10], Loss: 0.0766
Epoch [10/10], Loss: 0.0425
```

- Model Evaluation: A function calculated loss and accuracy on training and testing sets, revealing potential overfitting due to a significant difference in performance. A large gap exists between the training and testing metrics:

```
Training Set - Loss: 0.0409, Accuracy: 0.9944
Testing Set - Loss: 0.8251, Accuracy: 0.7833
```

#### 3.2-3.3 : F1 Score Evaluation

- Weighted F1 scores were calculated for both sets to assess performance with class imbalance.
- F1 Score Results: Both custom and torchmetrics implementations confirmed the performance, highlighting the drop-off in generalization:

```
--- Calculating F1 Scores for Training Set ---
Custom F1 Score (Weighted, Present Classes): 0.9945
TorchMetrics F1 Score (Weighted): 0.9945
Comparison: Custom F1 Score (0.9945) vs. TorchMetrics F1 Score (0.9945).

--- Calculating F1 Scores for Testing Set ---
Custom F1 Score (Weighted, Present Classes): 0.7671
TorchMetrics F1 Score (Weighted): 0.7671
Comparison: Custom F1 Score (0.7671) vs. TorchMetrics F1 Score (0.7671).
```

- Comparison: The identical results from the Custom F1 Score and torchmetrics.F1Score confirm the custom implementation's correctness for weighted F1 calculation.

### 3.4 : Confusion Matrix Analysis

- Confusion Matrix summary: A confusion matrix was generated for the test set.

```

--- Confusion Matrix for Test Set ---
-----
| Cropland | Tree Cover | Shrubland | Built-up |
-----
Cropland | 187 | 0 | 0 | 21 |
Tree Cover | 4 | 0 | 0 | 0 |
Shrubland | 1 | 0 | 0 | 1 |
Built-up | 25 | 0 | 0 | 1 |

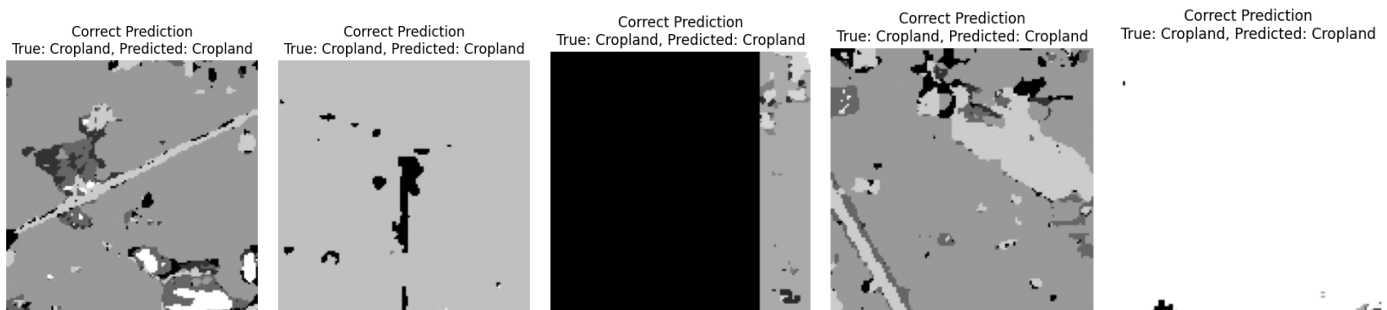
```

- Explanation : The model shows a strong bias toward Cropland (187 correct). It frequently misclassifies minor classes (Built-up (25), Tree Cover (4)) as Cropland, resulting in poor recall for non-dominant classes, a clear sign of severe class imbalance.

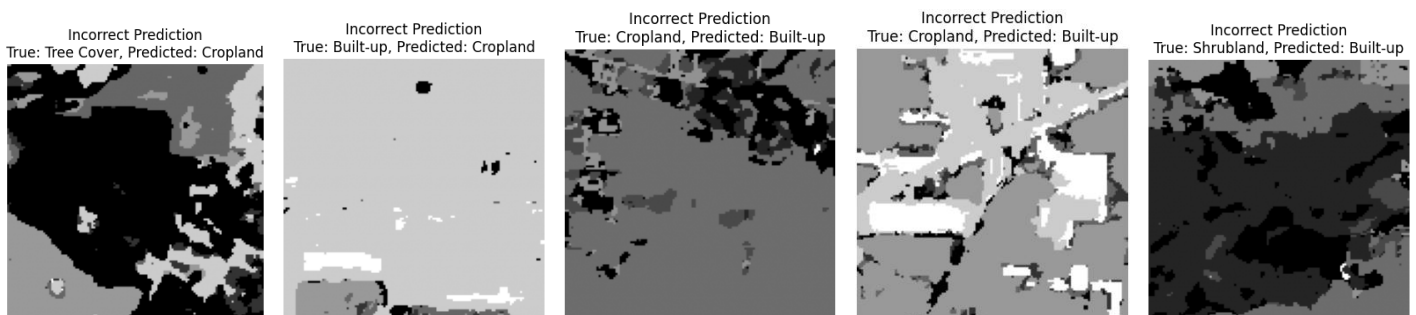
### 3.5 : Sample Predictions Visualization

- Visualization: A comparison plot of 5 correct and 5 incorrect test set predictions (with images and labels) was generated to qualitatively analyze model behavior and identify typical misclassification patterns

#### Visualization of 5 correct Predictions –



#### Visualization of 5 incorrect Predictions –



### Observations from Problem 3:

- The model exhibited significant overfitting, achieving high accuracy on the training set but considerably lower accuracy on the test set.

- The weighted F1 score provided a more informative performance metric than accuracy, highlighting the impact of the imbalanced dataset. The test set weighted F1 score was notably lower than the training set F1 score.
- The confusion matrix indicated that the model performed well on the dominant 'Cropland' class but struggled with other classes, frequently misclassifying them as 'Cropland'.
- Visualizing incorrect predictions offered valuable insights into the types of features or characteristics that led to misclassifications.

## Conclusion

This task successfully demonstrated a workflow for land cover classification in the Delhi-NCR region using satellite imagery and a CNN model. We were able to acquire and visualize the relevant geographic areas, extract land cover patches with corresponding labels, and build and train a basic CNN model.

The analysis revealed a significant class imbalance in the extracted dataset, with the 'Cropland' class being dominant. This imbalance likely contributed to the observed overfitting of the model, where it performed very well on the training data but less so on unseen test data, particularly for minority classes. The confusion matrix and F1 scores provided a more detailed view of the model's performance, highlighting its strength in classifying the dominant class and its weaknesses with less represented classes.